AI BUSINESS OPERATIONS ASSOCIATE · PRESENTATION ROUND

Golden Hour

An early-warning tool for project delivery

Mabel Nguyễn

Hackathon submission · 19/10/2026

Live app: golden-hour-pm.vercel.app

Code: github.com/Julie-hub-source/Golden_Hour_PM

About me

ABOUT ME

Nguyễn Cẩm Thạch - Mabel

Fresher Graduate of CS Major and Economics Minor

Current job: Junior IT Business Analyst at MoMo

Hobby: watching dramas, listening to music, eat everything



Agenda

AGENDA

What we'll cover

01 The challenge

02 Assumptions

03 The solution & live demo

04 How I built it with AI

05 Results & next steps

06 What I learned · Q&A

01

The challenge

THE HACKATHON PROMPT · THE EARLY WARNING SYSTEM

“Can you build a tool to surface where a project is really heading, and what to fix first, early enough to change the outcome?”

Status ≠ reality

Tools show what tasks should be doing, not what’s stuck

Hidden ripples

No one sees which delay will push everything after it

Too late

Managers find out at the deadline

Wrong detail

Leadership gets updates too detailed or too vague to act on

HOW I INTERPRETED IT

Three questions the tool must answer on one screen

1

Where is the project heading?

Projected Prod date vs. the client commitment → buffer

2

What's stuck, and who owns it?

Late tasks, blockers, owners and what depends on them

3

What should we fix first?

A ranked priority list, plus a what-if to test a delay

WHO IT'S FOR

Three users, three different needs

User	Needs	What Golden Hour gives them
PM	What to act on today	Priority list · what-if
Leadership / client	One clear signal	Buffer · Health (On track / Late)
Team members	Their own tasks	Task board · alerts · blockers

02

Assumptions

ASSUMPTION 1 · THE TEAM

Every project has all six roles

Role	Scope of work	Permissions
PM · Project Manager	Owns delivery and the client deadline; acts on the priority list	Everything: project settings, members, all tasks
PO · Product Owner	Owns scope; writes product docs, accepts each environment	All tasks (add, delete, deadlines, dependencies), client dates, description
BA · Business Analyst	Turns business needs into requirements; joins acceptance	Own tasks only: status, owner, duration, description, links, struggles 🔥
FE · Front-end Dev	Builds and fixes the user interface in UAT, STG and Prod	Own tasks only, same as BA
BE · Back-end Dev	Builds and fixes APIs, data and integrations	Own tasks only, same as BA
QC · Quality Control	Tests every environment; joins acceptance	Own tasks only, same as BA

Permissions are checked on the server, not just hidden in the UI · PO cannot change the start date, priority or members

ASSUMPTION 2 · THE DEFAULT FLOW

13 steps · 35 tasks · UAT → STG → Prod

Role (working days)

#	Step	Env	Tasks	Tasks
1	Prepare documents	UAT	BA – Business docs (3) · PO – Product docs (3)	2
2	Dev UAT	UAT	FE – Dev front-end (4) · BE – Dev back-end (5)	2
3	Test & Debug UAT	UAT	QC – Test (2) · BE – Debug (2) · FE – Debug (2)	3
4	Internal acceptance UAT	UAT	QC · BA · PO (1 each)	3
5	Customer acceptance UAT	UAT	QC · BA · PO (2 each)	3
6	Dev STG	STG	FE – Dev front-end (4) · BE – Dev back-end (5)	2
7	Test & Debug STG	STG	QC – Test (2) · BE – Debug (2) · FE – Debug (2)	3
8	Internal acceptance STG	STG	QC · BA · PO (1 each)	3
9	Customer acceptance STG	STG	QC · BA · PO (2 each)	3
10	Dev Prod	Prod	FE – Dev front-end (2) · BE – Dev back-end (2)	2
11	Test & Debug Prod	Prod	QC – Test (2) · BE – Debug (2) · FE – Debug (2)	3
12	Internal acceptance Prod	Prod	QC · BA · PO (1 each)	3
13	Customer acceptance Prod	Prod	QC · BA · PO (2 each)	3

ASSUMPTION 3 · THE RULES

What I decided, and why

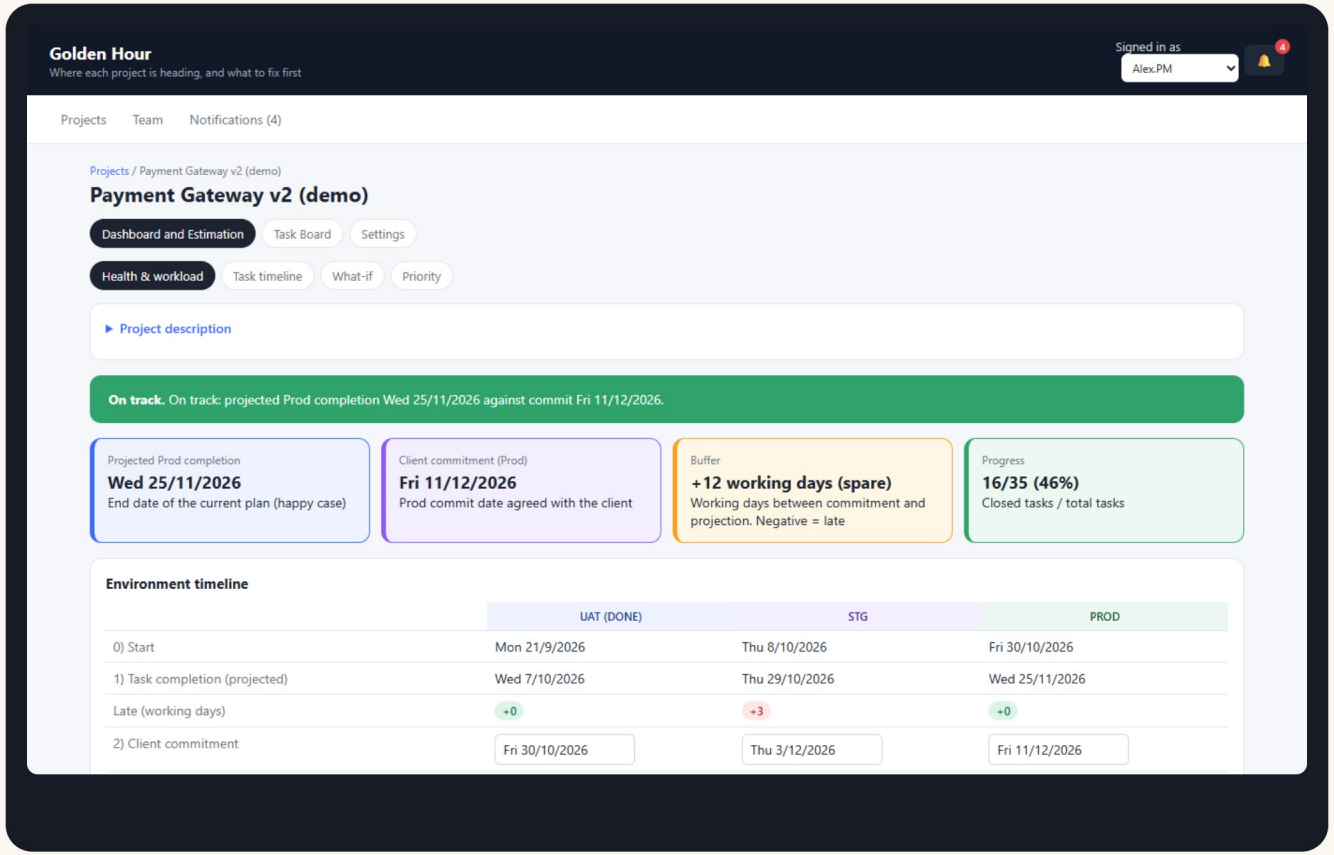
Assumption	Why
Working days only (Mon–Fri), no public holidays, Vietnam time	Simple, predictable date maths; holidays are an easy add-on
“Late” = not closed and past deadline	A task that couldn’t start yet is not its owner’s fault
Struggle = a blocker outside the team’s control, with a reason	Separates “we’re slow” from “we’re waiting”, so the PM knows who to chase
No login in the demo; switch user from a menu	Judges can test every role in seconds; permissions are still checked on the server
Outputs are links only (PRD, test evidence, PR...)	Teams already keep files in Drive or Jira; no duplicate storage

03

The solution

THE SOLUTION AT A GLANCE

One dashboard, four numbers that matter



Projected Prod
end date of the current plan

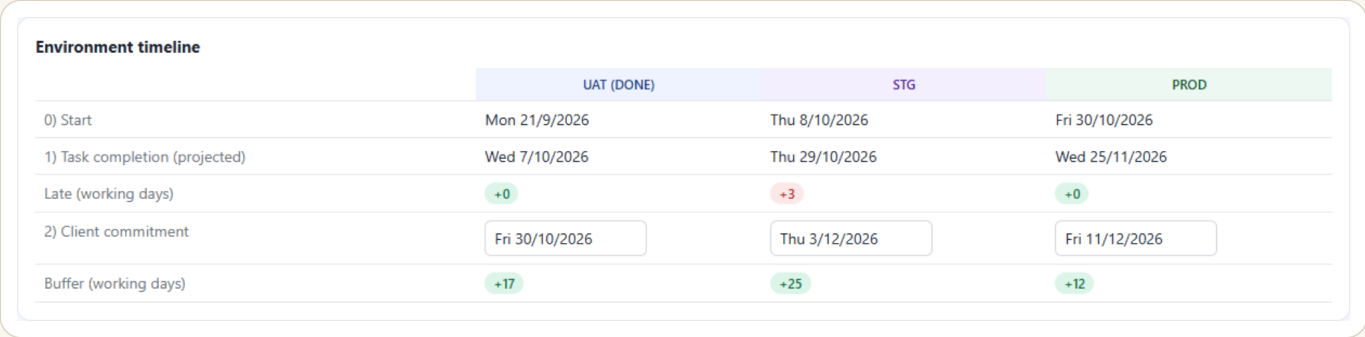
Client commitment
the date we promised

Buffer
working days between them; negative = late

Progress
closed tasks out of total

SCREEN · HEALTH & WORKLOAD

Is each environment on time, and is anyone overloaded?

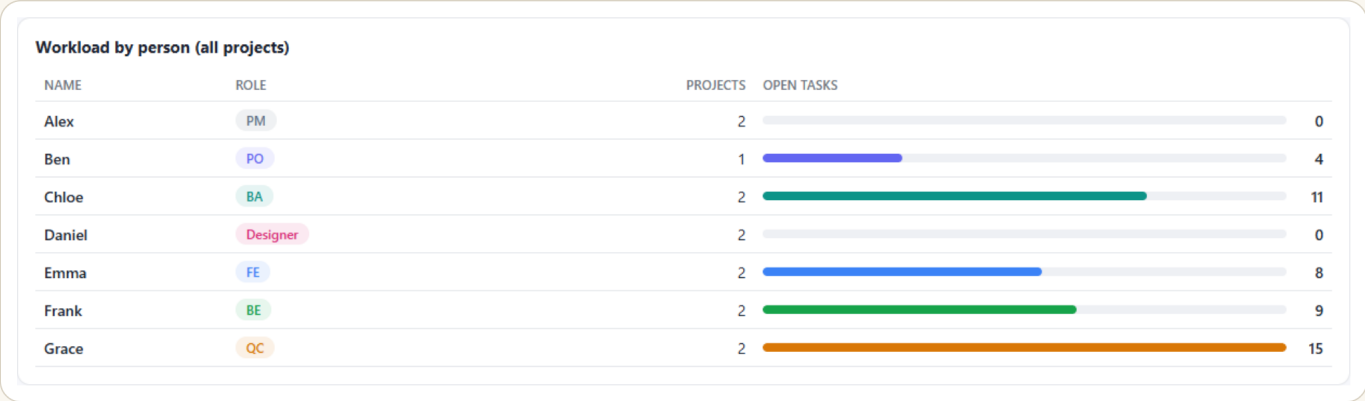


Environment timeline

	UAT (DONE)	STG	PROD
0) Start	Mon 21/9/2026	Thu 8/10/2026	Fri 30/10/2026
1) Task completion (projected)	Wed 7/10/2026	Thu 29/10/2026	Wed 25/11/2026
Late (working days)	+0	+3	+0
2) Client commitment	Fri 30/10/2026	Thu 3/12/2026	Fri 11/12/2026
Buffer (working days)	+17	+25	+12

① Environment timeline

For UAT, STG and Prod: start, projected finish, late days, the client date (editable), and the buffer. Green = spare days, red = late.



Workload by person (all projects)

NAME	ROLE	PROJECTS	OPEN TASKS
Alex	PM	2	0
Ben	PO	1	4
Chloe	BA	2	11
Daniel	Designer	2	0
Emma	FE	2	8
Frank	BE	2	9
Grace	QC	2	15

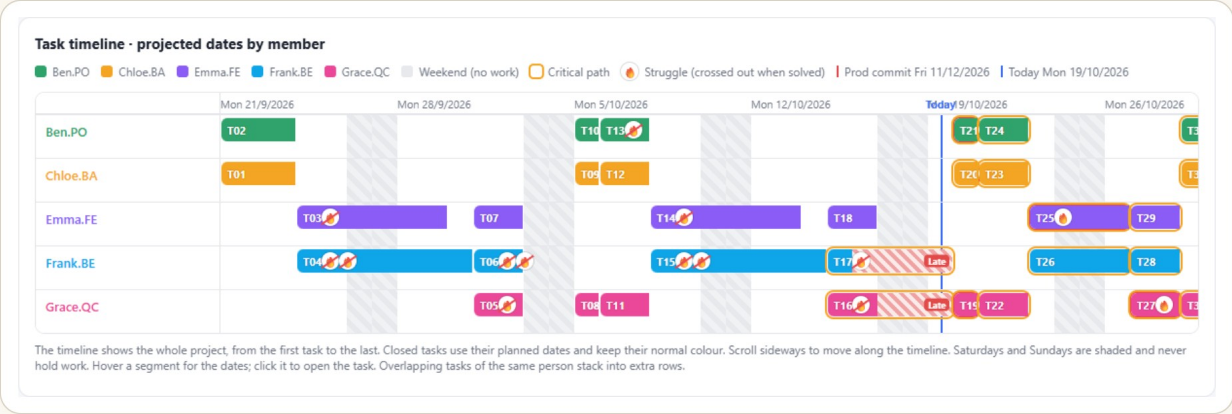
② Workload by person

Every member across all projects: role, projects, open tasks. Spot overload before it becomes a delay, e.g. Grace (QC) has 15 open tasks.

Above these: the health banner (On track / Late) and the four key figures from the previous slide

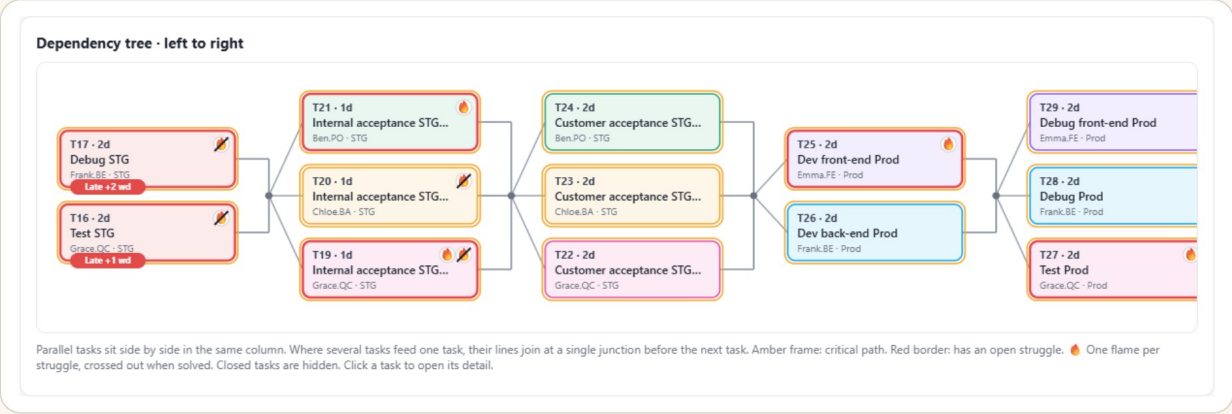
SCREEN · TASK TIMELINE

Who is doing what, and what waits on what



① Gantt chart

One lane per person, whole project. Red stripes + “Late” tag = late days. Amber outline = critical path. 🔥 = struggle. Blue line = today; red dashed line = Prod commitment.



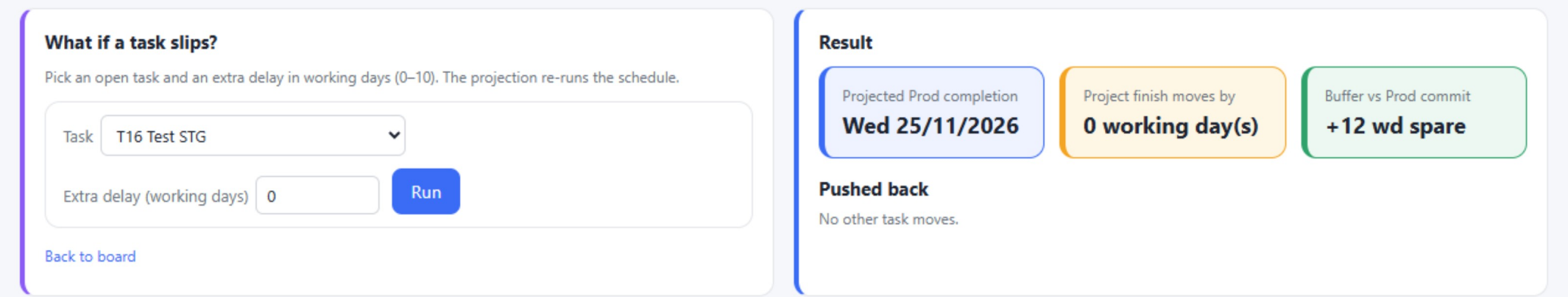
② Dependency tree

Tasks flow left to right; tasks in one column run in parallel. Late tasks show “Late +N wd”. You see how far one delay will ripple.

Click any bar or box to open the task detail

SCREEN · WHAT-IF

Test a delay before it happens



Input

Pick an open task and add 0–10 working days of delay

Output

New Prod finish, how far it moves, buffer vs. commitment, and which tasks get pushed

Use it when

Someone asks “can I have 3 more days?”, or before promising a date

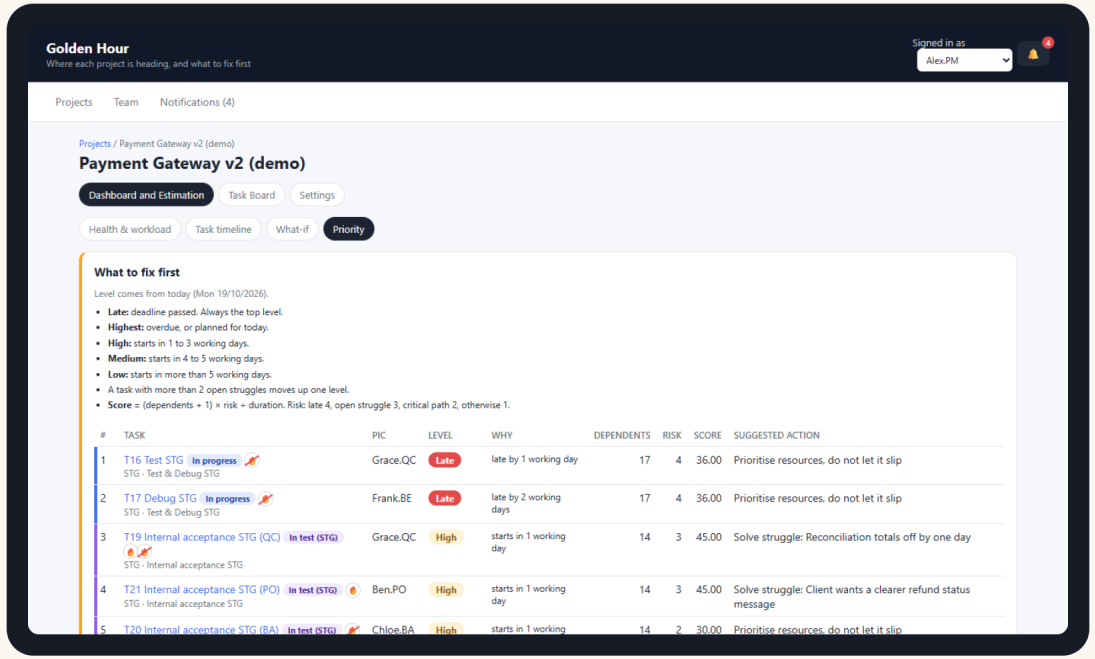
Nothing is saved: it only re-runs the schedule

SCREEN · PRIORITY — WHAT TO FIX FIRST

Five levels, set by today's date

Late	Deadline passed and the task has started. **Always on top.**
Highest	Planned end has passed, or it is planned for today
High	Starts in 1–3 working days
Medium	Starts in 4–5 working days
Low	Starts in more than 5 working days

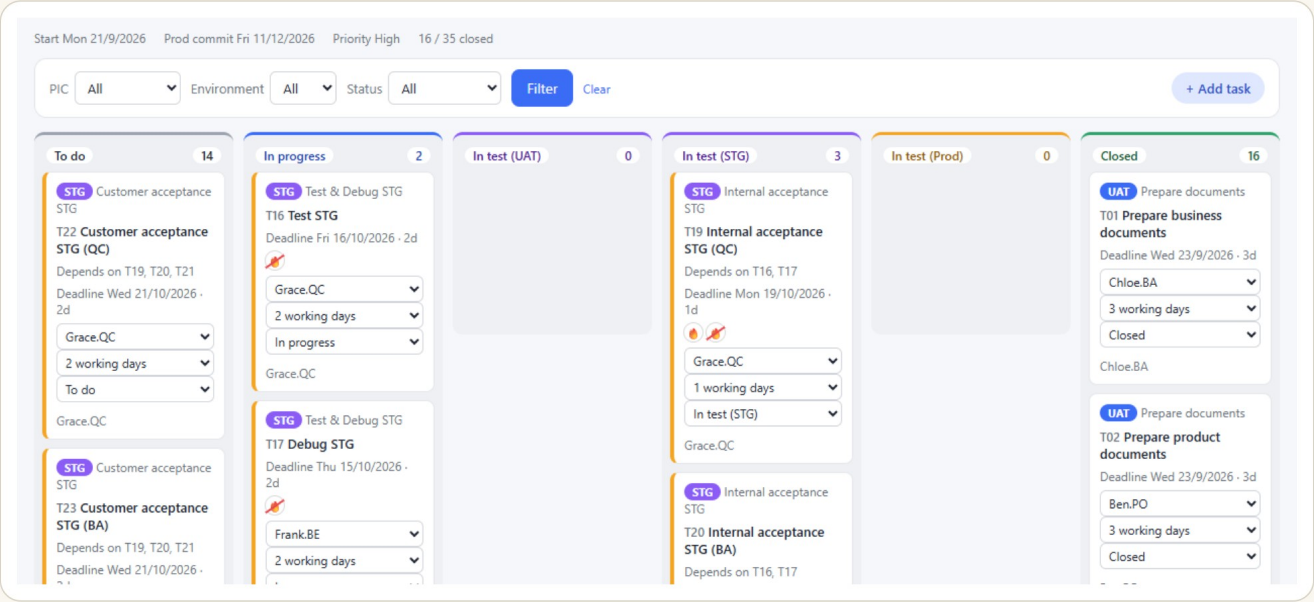
🔥 **Bump-up rule:** more than 2 open struggles moves a task up one level



Each row also shows the reason and a suggested action, e.g. “Solve struggle: vendor API not delivered”

SCREEN · TASK BOARD

Where the team updates its work every day



6 columns by status

To do · In progress · In test (UAT / STG / Prod) · Closed

Each card

Environment, deadline, depends on, owner, duration, status, one 🔥 per struggle

Quick updates

Change owner, duration or status on the card; members only on their own tasks

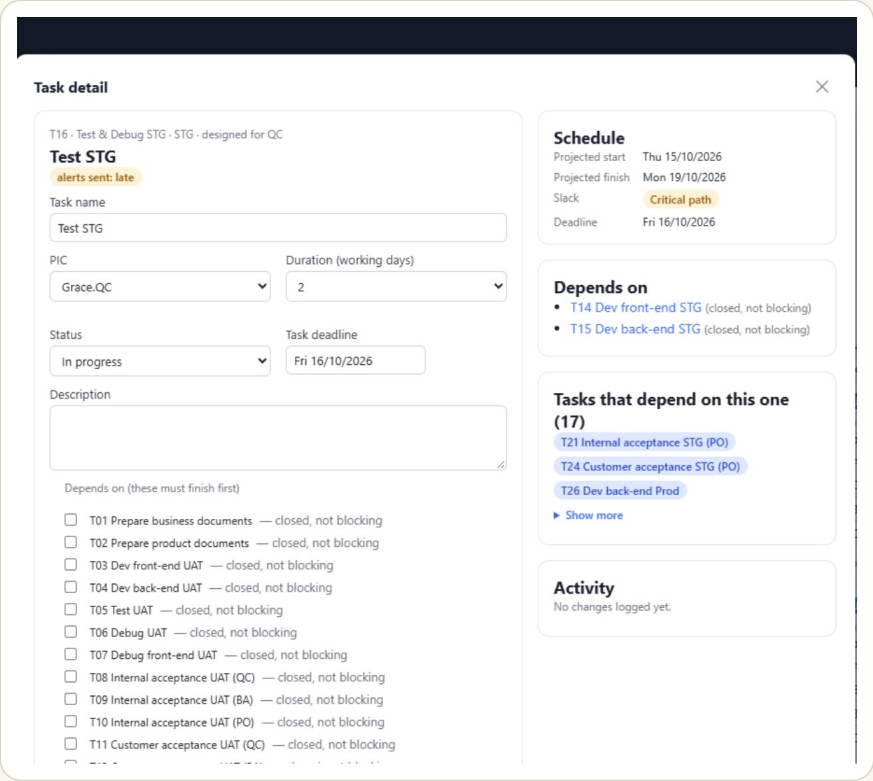
Filters

By owner, environment or status · “+ Add task” for PM and PO

Every update here feeds the dashboard straight away

SCREEN · TASK DETAIL

Everything about one task, in one popup



Edit

Name, owner, duration, status, deadline, description

Struggles 🔥

Add a blocker with a reason; mark it solved or reopen it

Output links

PRD, SRS, test evidence, design, code / PR...

Schedule

Projected start and finish, slack or “Critical path”

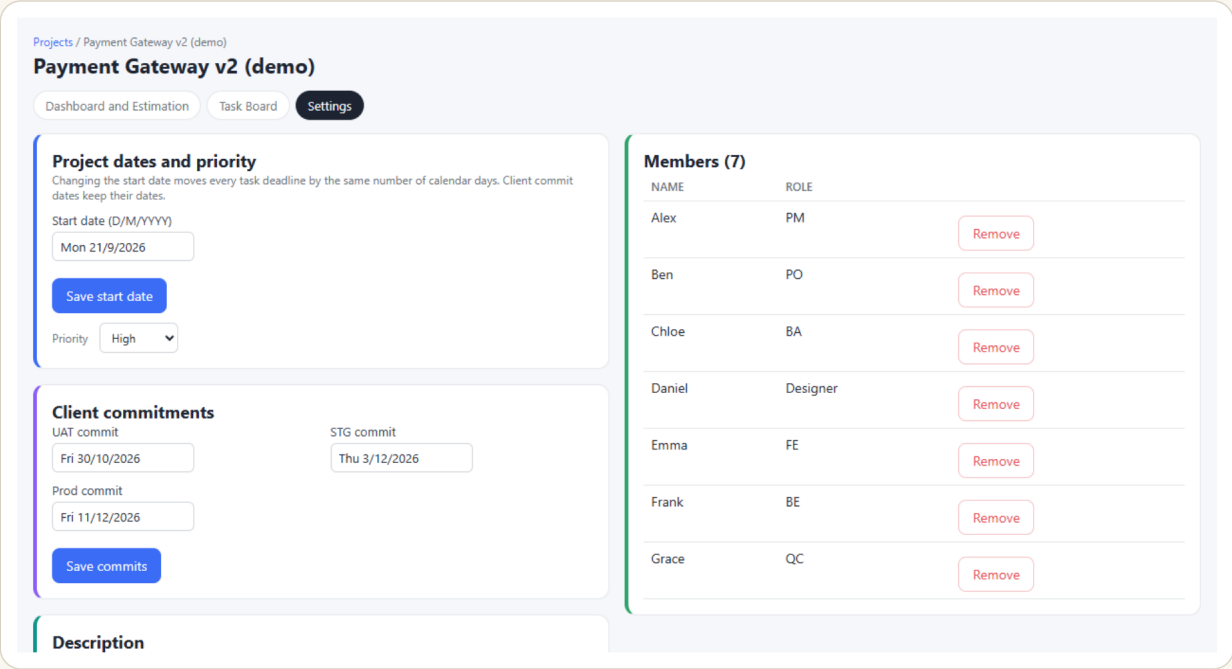
Dependencies, both ways

What it waits on, and what waits on it: 17 tasks for Test STG

Opens from the board, the Gantt or the dependency tree

SCREEN · SETTINGS

Dates, commitments and the team



Start date & priority · PM

A new start date moves every task deadline with it

Client commitments · PM, PO

UAT, STG and Prod dates the buffer is measured against

Description · PM, PO

Context, client info, related document links

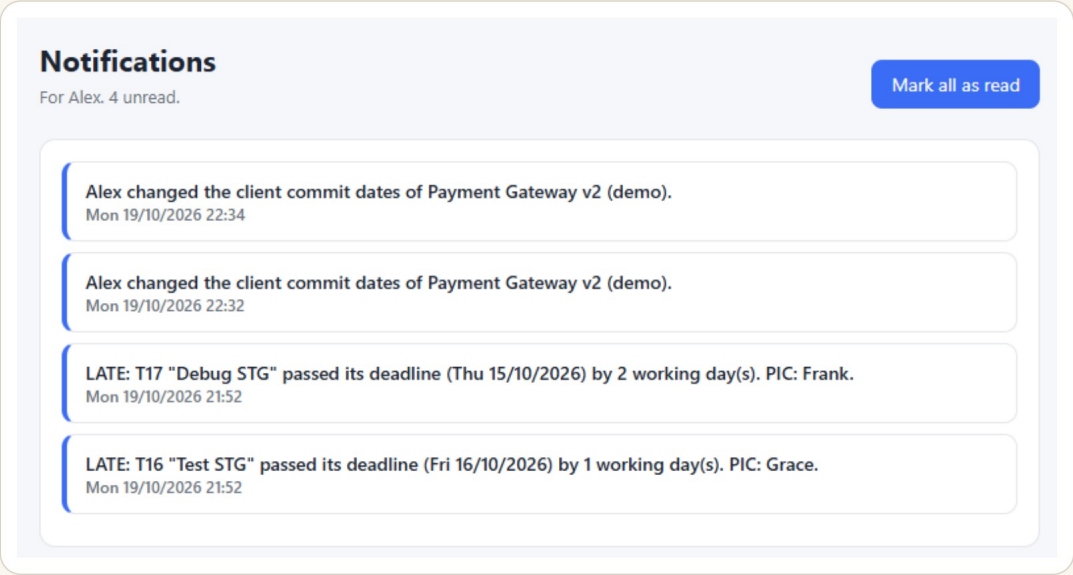
Members & delete · PM

Add or remove people; the demo project is locked against deletion

Each section shows who can change it; the server enforces the same rule

SCREEN · NOTIFICATIONS

The tool speaks up, so no one has to check



Two early-warning alerts

LATE: a task passed its deadline

To do too long: not started after 2 working days

When

Every day at 8:00, and when a project page opens; once per task until it changes

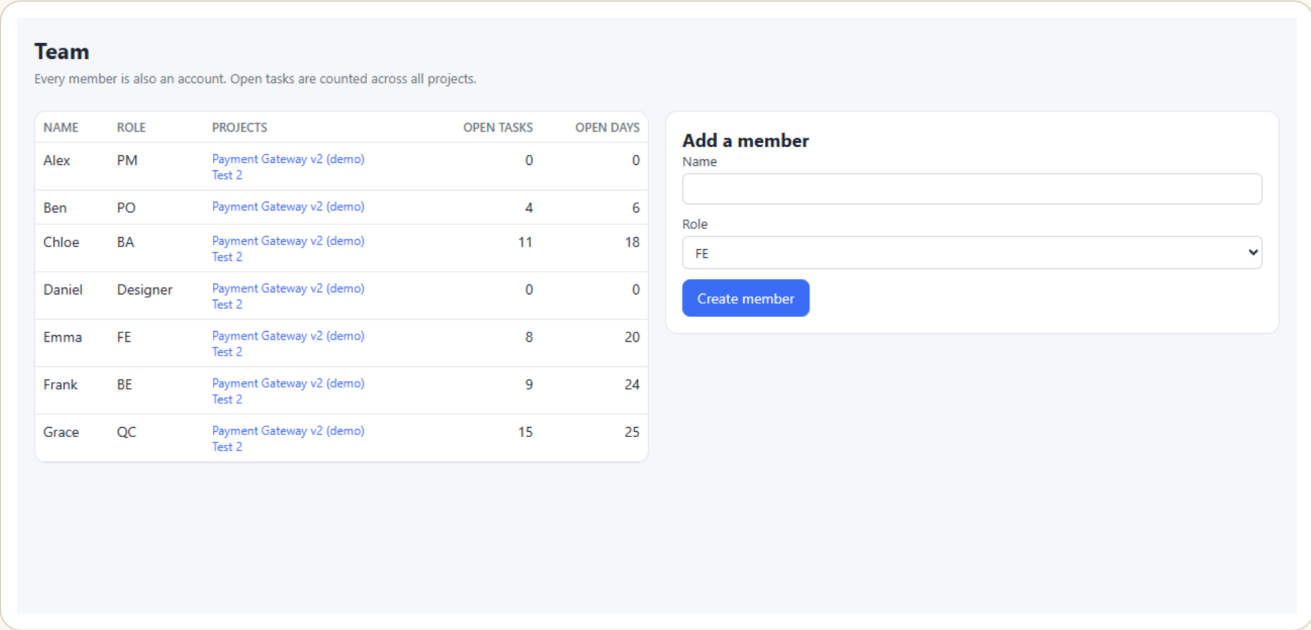
Who

Alerts → whole project. Task changes → PM, PO and the owner

In-app with a bell and unread count today · Slack, Teams and email are on the roadmap

SCREEN · TEAM

Capacity across all projects



Per member

Role, projects, open tasks and open working days

Add a member

PM only: name and role

Why it matters

Grace (QC) has 25 open days across 2 projects: the next bottleneck, visible before it hits

Every member is also an account in the “Signed in as” menu

FEATURE · TEST CLOCK

Pretend today is another date

TEST CLOCK: Mon 26/10/2026 Status changes are not saved. Closing the tab or turning the clock off returns to real time. **Change date · Exit**

Illustration of the bar shown in the header while the test clock is on

Why

Late rules and alerts depend on today. Without it, testing them means waiting days.

How

Pick any date in the header. All date logic uses it, and you can change task status to replay a scenario.

Safe

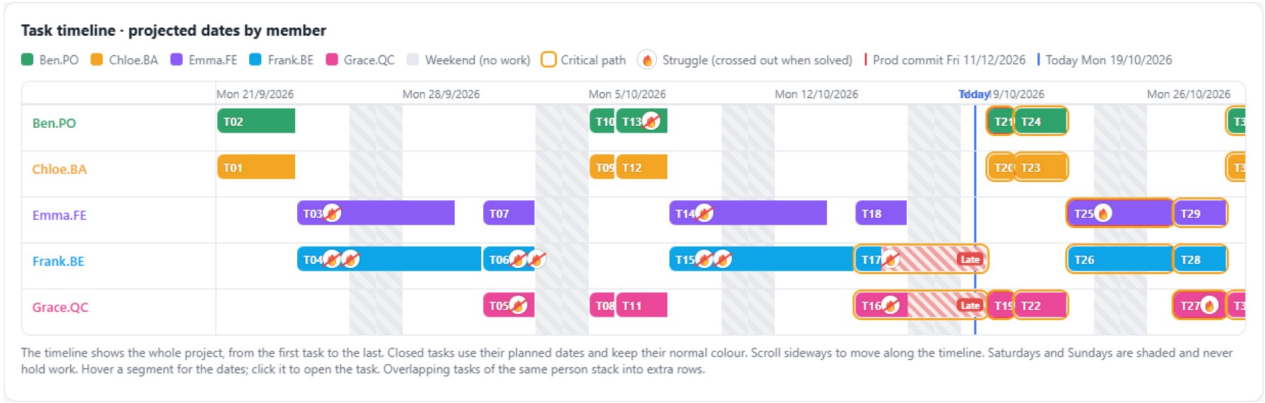
Kept in this browser only, never in the database: no notifications, no logs. A new tab is back to real time.

Added after the hackathon (1/11) to check the late and closed-task rules

LIVE DEMO · ONE STORY

“Back-end dev is blocked by a vendor API”

1. Add a 🔥 struggle: the task is flagged
2. Gantt: every task after it shifts later
3. Buffer turns negative, Health goes red
4. What-if: “+3 days?” shows the new finish
5. Priority list: what to fix first, and why
6. Switch to Emma.FE: she edits only her tasks



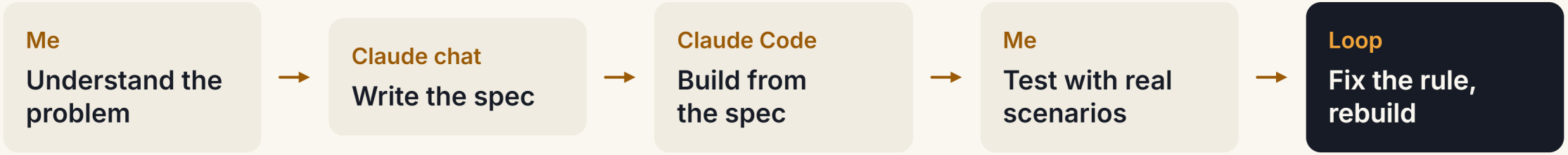
Demo: golden-hour-pm.vercel.app · project “Payment Gateway v2 (demo)”

04

How I built it

MY AI WORKFLOW

Spec first, then build, then test against real cases



Claude chat → the logic

Turned the business problem into rules: roles and permissions, the 35-task flow, scheduling formulas, late rules, alerts, edge cases. Output: **CLAUDE.md**, a 372-line spec.

Claude Code → the code

Read the spec and built the app: database, screens, permission checks, automated tests, deployment. When a number looked wrong, I fixed the rule first, then asked it to update the code.

Spec-first: the spec is the source of truth, and the code must match it

TECH STACK

Four simple parts, each chosen for a reason

SCREENS

Next.js + React

Dashboard, Gantt, task board, settings. Written in TypeScript.

Why: one codebase for screens and server logic; AI tools know it well.

LOGIC

Server actions

Scheduling engine, and every write. All permission rules live in one file.

Why: a hidden button can't be bypassed by a direct request.

DATA

Supabase (Postgres)

Projects, members, tasks, dependencies, struggles, notifications.

Why: task links fit a relational database; free tier; SQL I can read.

HOSTING

Vercel + Cron

Deploys on every push. Sends alerts daily at 8:00 Vietnam time.

Why: no server to manage; runs in Seoul, close to users.

Also used: GitHub · Automated tests (Node test runner) · Claude chat · Claude Code

Cost to run the demo: free tiers of Supabase and Vercel

TIMELINE & ITERATION

Live in 3 days, then kept improving

17/10

Idea, scope and spec

18/10

First working app, deployed

19/10

Design, docs, submitted

23/10

Reactivate supabase

1/11

Reactivate supabase + add clock for testing

3
days to a live app

27
commits

~5.3k
lines of code

372
lines of spec

05

Results & next steps

RESULTS

What works today, and what doesn’t yet

Delivered

- Live, deployed app anyone can try
- Answers all 3 questions on one screen
- Role permissions checked on the server
- Daily alerts for late and stuck tasks
- User guide + full spec for maintainers

Limitations

- No real login yet
- No public holidays
- Alerts are in-app only
- Tested on demo data, not a live project
- No UT
- Fixed roles: new roles can’t be added freely
- Fixed flow: no urgent or hot-fix tasks
- No overlap, e.g. docs and dev in parallel

WHAT'S NEXT

From demo to a tool a real team uses

1

Real sign-in

Higher security; each person sees only their projects

2

Jira sync

Import tasks first, then two-way sync, so no one types twice

3

Alerts where people are

Slack, Teams and a daily email digest

4

Weekly snapshots

Is the project getting better or worse over time?

06

What I learned

WHAT I LEARNED

Three lessons I'll bring to the next tool

1 A clear spec makes AI far more accurate

2 Testing with real scenarios catches what the AI can't see

3 Ship early, then iterate on what you learn

Q&A

Thank you

Happy to go deeper on any decision

Mabel Nguyễn

Live app: golden-hour-pm.vercel.app

Code: github.com/Julie-hub-source/Golden_Hour_PM